Expert Systems With Applications 299 (2026) 130014



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





# An acoustic sensing system for noise monitoring and source identification using transfer learning

Dolvara Gunatilaka, Wudhichart Sawangphol *, Thanakorn Charoenritthitham, Thanawat Kanjanapoo, Teerapat Burasotikul, Kittikawin Pongprasit

*Faculty of Information and Communication Technology, Mahidol University, Thailand*

ARTICLE INFO

*Keywords:*
Urban noise classification
Transfer learning
Acoustic sensing

ABSTRACT

Increasing noise pollution in urban areas underscores the need for an autonomous system to monitor and control noise. Beyond detecting noise levels, identifying noise sources further improves noise management. This work presents a scalable IoT-based sensing platform for smart environment applications. The system integrates low-cost devices for acoustic measurement, edge devices to enable noise source identification, a back-end infrastructure crucial for efficient acoustic data and device management, and a web-based application facilitating noise data visualization. Our study explores three feature extraction techniques and eight Convolutional Neural Network (CNN)-based pre-trained models for noise classification on the resource-constrained Raspberry Pi platform and compares their performance. Leveraging pre-trained models helps speed up the model development process. UrbanSound8k, ESC-50 datasets, and audio data collected with our low-cost microphone are used for model development and validation. The evaluation results show that our hierarchical model, utilizing the Mel Spectrogram feature extraction method and a MobileNet model, achieves the highest accuracy of 90.18 %. Furthermore, we deploy the system and assess its performance. Our system can reliably transmit audio data with an average delay of 0.37 s, and the Raspberry Pi can perform feature extraction and classification within an average of 2.5 s. Hence, our solution offers a comprehensive and cost-effective solution to enhance noise management and control.

## 1. Introduction

Noise pollution has a significant impact on the well-being of urban residents, contributing to various health issues. Studies (Basner et al., 2014; Mohamed et al., 2021) confirm the detrimental effects of noise exposure on both physical and mental health. Due to the dynamic nature of the urban acoustic environment, continuous and long-term acoustic scene monitoring is necessary for effective noise management. IoT technology enables the development of noise monitoring networks by distributing sensors across urban areas, providing a scalable and affordable solution (Alías & Alsina-Pagès, 2019; Picaut et al., 2020). These systems typically measure sound pressure levels (SPL), indicating the loudness of sound, and rely on cloud or local servers to gather and display noise data. However, measuring noise level alone is insufficient for effective noise control since it offers a limited view of the problem, lacking details of the noise event. Identifying the noise source is essential for targeting mitigation efforts and strategies. It allows urban planners, environmental agencies, and residents to address noise issues more efficiently. Common sources of noise pollution include transportation, construction, and entertainment activities (Mohamed et al., 2021). As a result, many projects (AbeBer et al., 2018; Arce et al., 2021; Bello et al., 2019; Maijala et al., 2018; Siamwala et al., 2019; Vidaña-Vila et al., 2020) integrate classification models to detect and identify these diverse noise sources automatically.

Environmental sound classification enables automatic detection and categorization of noise events. Deep learning techniques, particularly Convolutional Neural Networks (CNN), are frequently adopted for noise source identification (Bansal & Garg, 2022). To expedite the model development process, some researchers (Aksoy et al., 2022; Ashurov et al., 2022; Tsalera et al., 2021; Vidaña-Vila et al., 2020) leverage transfer learning by using pre-trained models rather than implementing them from scratch. While deep learning models excel at capturing complex audio signal patterns and can achieve high accuracy in audio classification, they are computationally costly and require powerful central servers. This approach involves transferring audio data to the central server, which can cause a communication bottleneck and restrict system scalability. Alternatively, many acoustic sensing systems adopt a distributed architecture, where noise classification is performed locally

* Corresponding author.
*E-mail address:* wudhichart.saw@mahidol.ac.th (W. Sawangphol).

https://doi.org/10.1016/j.eswa.2025.130014
Received 31 July 2025; Received in revised form 1 October 2025; Accepted 11 October 2025
Available online 15 October 2025
0957-4174/© 2025 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

on embedded or edge devices such as the Raspberry Pi. This presents a challenge due to the limited resources available on these devices. As a result, deploying deep learning models on such platforms requires optimization techniques, e.g., model compression and quantization, to minimize model size and computational demands.

This work proposes an IoT-based distributed acoustic monitoring network designed for continuous noise level monitoring and noise source identification. Our work investigates the effectiveness of pre-trained CNN models in addressing the noise classification challenge on edge devices. The chosen noise analysis model is deployed on a Raspberry Pi, which is integrated into the low-cost acoustic sensing network. This solution offers a scalable and cost-effective approach to intelligent noise pollution monitoring and classification.

The contribution of this study can be summarized as follows.

- We develop a CNN model using transfer learning for resource-constrained devices, utilizing open-source datasets and audio data collected with our low-cost microphone. We focus on ten classes of common urban noise comprising drilling, jackhammers, street music, chainsaws, car horns, car engines, trains, airplanes, handsaws, and other sounds.
- We conduct an empirical study to investigate the performance of three audio feature extraction techniques, namely, Spectrogram, Mel-Frequency Cepstral Coefficient (MFCC), and Mel Spectrogram, and eight pre-trained CNN models: MobileNet, MobileNetV2, MobileNetV3Small, MobileNetV3Large, EfficientNetB1, EfficientNetV2B1, VGG16, and VGG19. Furthermore, we explore alternative model implementations, including an image-based approach where audio data is transformed into images for classification, as well as a hierarchical model design. These models are quantized and evaluated in terms of accuracy and size.
- We design and implement a comprehensive acoustic sensing platform that automates noise level measurement and source classification. The system consists of (1) an acoustic sensing device equipped with a low-cost microphone for measuring sound pressure level and recording noise events, (2) an edge device (Raspberry Pi) that locally classifies noise, and (3) a robust back-end infrastructure, which is critical for audio data aggregation and sensing device management, and (4) a web dashboard that visualizes real-time and historical noise level and events. It is then evaluated in terms of inference time, model accuracy, data transmission reliability, and delays.

The rest of the paper is organized as follows. Section 2 reviews related works. Section 3 presents the noise source identification model development. Section 4 includes experiments to assess model performance. Section 5 describes the development of the acoustic sensing system. Section 6 provides system evaluation results, and Section 7 concludes this work.

## 2. Related works

In this section, we review works related to environmental sound classification and wireless sensor networks for noise monitoring and classification.

### 2.1. Environmental sound classification

Noise monitoring systems utilize machine learning techniques to develop a sophisticated framework for recognizing environmental sounds. Deep learning techniques, especially *Convolutional Neural Networks (CNN)*, are adopted because of their ability to learn complex representations of audio signals and perform well in audio classification (Bansal & Garg, 2022). Many studies use methods such as Spectrogram and Mel-Frequency Cepstral Coefficients (MFCCs) to capture key acoustic characteristics and produce compact audio representations. This helps reduce the model's complexity and speeds up the training process. UrbanSound8K (Salamon et al., 2014), ESC-10, and ESC-50 (Piczak, 2015b) are three notable datasets used for training and developing models.

Earlier work from Piczak (Piczak, 2015a) demonstrates the effectiveness of a 2D CNN using spectrograms for acoustic data classification. Salamon and Bello (2017) augment their CNN framework by introducing multiple data augmentation methods to increase data diversity, thus attaining a model's accuracy of 79 %. Vujošević and Dukanović (2021) also propose a CNN architecture that integrates multiple feature extraction techniques, including Mel Spectrogram, Chromagrams, Tonal Centroids, and Spectral Contrasts to transform audio signals into image inputs. Their model realizes an accuracy of 73 %. In contrast, Abdoli et al. (2019) introduce a 1D CNN model that directly learns and optimizes features from the audio waveforms, obtaining an average accuracy of 89 %. Furthermore, attention mechanisms have been integrated into CNNs to improve their ability to focus on crucial audio features (Ashurov et al., 2024; Wang et al., 2021; Yang et al., 2023).

Pre-trained CNN models have been increasingly explored as they offer robust performance in audio classification and help speed up the model development process. Aksoy et al. (2022) evaluate three very deep CNN architectures, namely, VGG-13BN, ResNet-50, and DenseNet-121. All three architectures demonstrate high effectiveness with over 97 % accuracy, but incur high computational requirements. Ashurov et al. (2022) investigate nine pre-trained models, which are fine-tuned and applied with different optimization techniques to ensure accuracy. DenseNet-201 with Adam optimizer yields the best performance at 97 % accuracy. Additionally, Tsalera et al. (2021) compare the performance of five transfer learning models (GoogLeNet, SqueezeNet, ShuffleNet, VGGish, and YAMNet). Their findings indicate an average prediction accuracy of 96.4 %. While these studies primarily emphasize exploring different machine learning algorithms and techniques to enhance classification performance, they do not specifically address deployment considerations for embedded or edge platforms.

Increasing interest has shifted towards the development of sound classification models on resource-constrained devices. Although some studies (Alsouda et al., 2019; da Silva et al., 2019) use supervised machine learning algorithms due to their lightweight nature, several projects opt to explore CNN-based models. Mohaimenuzzaman et al. (2023) introduce ACDNet, a CNN-based model, and its compression pipeline that produces a compact model optimized for edge deployment. This work attains classification accuracy of 86.65 % and 89.69 % based on the UrbandSound8K and ESC-50 datasets, respectively. Lamrini et al. (2023) assess the performance of pre-trained 1D CNN and ANN models on two embedded platforms, i.e., Raspberry Pi and USB Coral TPU, with three different datasets. Furthermore, Peng et al. (2023) propose a classification system using CNN-LSTM on a Jetson Nano board that can achieve 94.94 % accuracy, and an execution time of 35 ms for 6-s audio data.

The aforementioned projects largely focus on developing and evaluating classification models on a stand-alone edge device, but do not address the implementation of the sensing network and its infrastructure. Our work includes a comparative analysis of a wide range of feature extraction techniques and pre-trained CNN models and an assessment of model performance on edge devices. The model with the best performance is then integrated into our acoustic sensing network.

### 2.2. Wireless sensor networks for noise monitoring and classification

Several related projects aim to develop acoustic sensing networks for environmental noise monitoring, which in turn support smart city initiatives. These works involve designing and implementing low-cost acoustic sensing devices, their communication infrastructure, and the back-end system for storing, analyzing, and visualizing audio data.

Studies (Bartalucci et al., 2018; Mydlarz et al., 2017) focus on implementing acoustic sensors capable of capturing and processing audio signals to determine the equivalent continuous sound level ($L_{Aeq}$).

These works aim to achieve accurate sensor measurement, sensor resiliency, and energy efficiency. Several projects (Bellucci et al., 2017; Fernandez-Prieto et al., 2020; Rainham, 2016) seek to monitor noise pollution across various urban areas. Beyond building the acoustic sensing network, these works also involve deploying a central server that aggregates noise data and analyzes it to generate noise maps. A web dashboard then provides real-time and historical noise data visualization.

Additionally, many efforts enhance their noise monitoring system by incorporating intelligent noise event detection and identification algorithms. Siamwala et al. (2019) propose a system comprising a Bluetooth Low Energy (BLE) node for acoustic sensing, and a Raspberry Pi as a central processing unit, which computes sound pressure level and classifies noise sources. Maijala et al. (2018) utilize Raspberry Pi as a sensing device for similar tasks. Both works adopt traditional machine learning techniques, including the Gaussian Mixture Model (GMM) and the Artificial Neural Network (ANN), which require less computational workload.

Conversely, there are existing works that integrate deep learning models into their sensing networks. Vidaña-Vila et al. (2020) compare pre-trained CNN model performance and propose a distributed consensus protocol to improve the accuracy of the local acoustic event classification. Shah et al. (2019) implement a noise monitoring system leveraging 2D CNN model for urban sound classification on Raspberry Pi devices. Their classification results are transmitted and logged in the Google Firebase cloud. Arce et al. (2021) develop a sensing network capable of recognizing four sound events using a CNN-based model. This work relies on FIWARE (Foundation, 2025), an open IoT platform for smart cities, to facilitate communication between sensing devices and the cloud server. A Grafana dashboard is employed to display noise data. Likewise, AbeBer et al. (2018) deploy a CNN classifier on the sensing device to enable noise source identification. Their implementation includes a centralized server with an administrative component to accommodate sensor device configuration with a web application that displays noise maps and noise-including public events.

Our work covers the design and implementation of all layers of the distributed sensing system for noise monitoring and identification, ranging from the low-cost acoustic sensing devices, the edge devices for noise classification using a pre-trained CNN model, the back-end infrastructure, and the web-based application. Our web application provides comprehensive features. It displays current noise levels and events on a map, and presents historical noise data for future analysis. It allows users to submit a noise complaint, which can be correlated with events automatically detected by our sensing platform. Moreover, to enable effective device management, our system supports sensing device authentication and configuration necessary for system maintenance.

## 3. Noise source classification model development

This section describes the model development process, which comprises data collection and pre-processing, as well as model implementation and optimization.

### 3.1. Data collection and pre-processing

**Data Collection:** To develop our noise source identification model, we leverage open-source datasets, namely, UrbanSound8K (Salamon et al., 2014) and ESC-50 (Piczak, 2015b), and obtain additional audio clips from Pixabay. To enhance model robustness, we supplement this data with manually collected recordings using an INMP441 low-cost microphone (Invensense, 2025). Here, audio samples were captured at a 16 kHz sampling rate with 16-bit depth. We focus on ten common noise pollution sources, including *airplanes*, *car engines*, *car horns*, *chainsaws*, *drilling*, *handsaws*, *jackhammers*, *street music*, *trains*, and *other urban sounds* (i.e., thunderstorms, fireworks, gunshots, sirens, crying babies, and barking dogs). Each audio sample is approximately five seconds long in the WAV format. Table 1 summarizes the number of audio samples acquired from each source based on ten noise classes. Our dataset consists of a total of 1290 audio samples. To prevent bias and to ensure model generalization, we try to balance the number of samples in each class.

**Table 1**
The number of audio samples categorized based on classes and sources.

| Class | Urban Sound8K | ESC-50 | Pixabay | Manual Record with INMP441 | Total |
|---|---|---|---|---|---|
| Airplane | – | 40 | 20 | 50 | 110 |
| Car | – | 20 | – | 100 | 120 |
| Car Horn | 100 | – | – | 40 | 140 |
| Chainsaw | – | 40 | – | 100 | 140 |
| Drilling | 80 | – | – | 60 | 140 |
| Handsaw | – | – | – | 80 | 120 |
| Jackhammer | 80 | – | – | 60 | 140 |
| Street Music | 80 | – | – | 60 | 140 |
| Train | – | 40 | 10 | 60 | 110 |
| Others | 90 | 90 | – | 22 | 130 |

**Data Augmentation:** We increase dataset diversity and sample size to reduce model overfitting by employing the following augmentation techniques.

- Background Noise: We introduce artificial white noise, a random signal with a uniform spectral density across all frequencies, to an audio sample.
- Pitch Shifting: We increase the pitch of the audio sample by performing time stretching and adjusting the frequency components of the audio.

Through data augmentation, we increase the number of samples by threefold, resulting in the final dataset of 3870 samples.

**Feature Extraction:** Next, the feature extraction technique is applied to convert raw audio data into a compact representative feature set that captures important audio characteristics. We investigate three widely adopted methods for audio feature extraction.

- **Spectrogram** represents audio data in the time and frequency domains. This is achieved by applying a Short-Time Fourier Transform (STFT) on the audio signal.
- **Mel Spectrogram** is a variation of Spectrogram in which the output spectrum of STFT is mapped to a Mel scale; a scale of pitch that aligns with how human ears respond to different audio frequencies.
- **Mel-Frequency Cepstral Coefficients (MFCC)** are derived by applying the Fast Fourier Transform (FFT) and a Mel-Scaled filter to the audio signal and employing the Discrete Cosine Transformer (DCT) to obtain a compressed representation of spectral characteristics.

Since the pre-trained models are originally designed for image data, we transformed the audio features into a suitable format. A 2D time-frequency representation of the audio signal was computed using a Mel spectrogram. The resulting matrix (frequency $\times$ time) is then expanded into a 3D format by adding a channel dimension. To match the input requirement of CNNs, the spectrogram is resized to a fixed resolution of 128 $\times$ 128. We construct a three-channel representation by duplicating the single-channel spectrogram across the three channels, which is similar to the process of replicating a grayscale image into RGB channels. Thus, the final target input shape is (128, 128, 3). Although many pre-trained architectures commonly use 224 $\times$ 224 inputs, we opt for 128 $\times$ 128 to reduce computational costs and memory usage.

### 3.2. Model implementation and optimization

We leverage pre-trained convolutional neural network (CNN) architectures using the TensorFlow framework and Keras API to accelerate

training and improve generalization. The models explored in this study include *MobileNet*, *MobileNetV2*, *MobileNetV3Small*, *MobileNetV3Large*, *EfficientNetB1*, *EfficientNetV2B1*, *VGG16*, and *VGG19*. These architectures were selected based on their proven performance on image classification tasks and their varying trade-offs between model size and accuracy, which are particularly important for deployment on resource-constrained environments.

The dataset is partitioned into training (80 %), validation (10 %), and testing (10 %) subsets, with stratified sampling to ensure class balance across all splits. To ensure reproducibility, we fix the random seed at 42 during data splitting and all subsequent training procedures. Each pre-trained model is used as a base feature extractor, with its convolutional layers initially frozen to retain the learned representations from large-scale image datasets such as ImageNet.

We construct a sequential model that utilizes the pre-trained model as a base layer and then adds custom layers, such as Dropout and Dense layers, to tailor the model for our classification task. The models are compiled using the Adam optimizer, set with an initial learning rate of 0.001 and a batch size of 32. Training is conducted for a maximum of 100 epochs. To prevent overfitting and save computational costs, early stopping is implemented by monitoring the validation loss, with a patience of five epochs. This means training will be halted if the performance does not improve for five consecutive epochs. These hyperparameters were established through preliminary experiments and are consistent with values found in relevant literature.

After the training phase, we implement post-training quantization to optimize the model for deployment on resource-limited devices. Using the TensorFlow Lite framework, we perform full integer quantization, which involves converting the model weights and activations from 32-bit floating-point format to 8-bit integers. This process significantly reduces the model's size and inference latency while resulting in minimal degradation in classification accuracy.

The final model performance is evaluated using the unseen test set. Evaluation metrics include overall accuracy and loss, which provide a comprehensive view of model effectiveness across all classes.

## 4. Model evaluation and results

In this section, we conduct empirical studies to determine the optimal model for our noise source identification system. We conduct the following experiments.

- Experiment 1: Conduct a comparative study to identify the combination of feature extraction technique and pre-trained model that achieves the maximum performance.
- Experiment 2: Assess the impact of quantization on model performance, and determine feasibility for deployment on edge or sensing devices.
- Experiment 3: Explore image-based and hierarchical models.

### *4.1. Experiment 1: Conduct a comparative study to identify a combination of feature extraction techniques and a pre-trained model that achieves the maximum performance.*

In this experiment, we follow the steps in Section 3 to develop and train the model (excluding the quantization step). We evaluate the model's performance using common metrics, including accuracy and loss. Table 2 compares the performance of 24 models (3 feature extractions x 8 model architectures) based on the training, validation, and test datasets. For brevity, MobileNetV3Small and MobileNetV3Large are referred to as MobileNetV3S and MobileNetV3L, respectively.

The results demonstrate that Mel Spectrogram, combined with any model architecture, consistently outperforms Spectrogram and MFCC. In contrast to the Spectrogram, both MFCC and Mel Spectrogram utilize the Mel scale, which focuses on frequencies sensitive to human auditory perception. While MFCC generates a time-independent and compressed audio representation, Mel Spectrogram offers detailed time and frequency information on the audio signal. This richer representation could potentially enhance CNN's ability to discriminate between different sound types.

**Table 2**
Model accuracy and loss under training, validating, and test sets.

| Feature Extraction | Model Architecture | Train | | Validate | | Test |
|---|---|---|---|---|---|---|
| | | Accuracy | Loss | Accuracy | Loss | Accuracy |
| Spectrogram | MobileNet | 58.55 | 2.40 | 61.76 % | 2.36 | 66.67 % |
| Spectrogram | MobileNetV2 | 57.27 | 2.66 | 62.53 % | 2.50 | 63.05 % |
| Spectrogram | MobileNetV3S | 61.14 % | 2.78 | 63.31 % | 2.74 | 69.77 % |
| Spectrogram | MobileNetV3L | 66.45 % | 2.80 | 71.58 % | 2.70 | 72.09 % |
| Spectrogram | EfficientNetB1 | 38.06 % | 2.46 | 40.83 % | 2.35 | 47.80 % |
| Spectrogram | EfficientNetV2B1 | 50.13 % | 2.34 | 56.85 % | 2.21 | 59.43 % |
| Spectrogram | VGG16 | 62.83 % | 2.33 | 71.06 % | 2.16 | 70.28 % |
| Spectrogram | VGG19 | 59.99 % | 2.29 | 73.39 % | 2.07 | 71.06 % |
| Spectrogram Mel | MobileNet | 79.87 % | 2.00 | 86.30 % | 1.81 | 87.60 % |
| Spectrogram Mel | MobileNetV2 | 74.41 % | 2.05 | 78.81 % | 1.94 | 79.84 % |
| Spectrogram Mel | MobileNetV3S | 71.94 % | 2.86 | 75.19 % | 2.81 | 78.55 % |
| Spectrogram Mel | MobileNetV3L | 77.03 % | 3.11 | 79.33 % | 2.99 | 82.43 % |
| Spectrogram Mel | EfficientNetB1 | 49.31 % | 2.35 | 63.05 % | 2.11 | 67.96 % |
| Spectrogram Mel | EfficientNetV2B1 | 57.46 % | 7.36 | 65.37 % | 7.83 | 68.73 % |
| Spectrogram Mel | VGG16 | 64.06 % | 4.59 | 74.16 % | 4.20 | 72.87 % |
| Spectrogram | VGG19 | 64.05 % | 4.54 | 73.90 % | 4.15 | 79.84 % |
| MFCC | MobileNet | 67.31 % | 2.23 | 68.99 % | 2.15 | 76.23 % |
| MFCC | MobileNetV2 | 46.21 % | 2.29 | 58.40 % | 2.05 | 58.91 % |
| MFCC | MobileNetV3S | 61.25 % | 1.96 | 67.70 % | 1.84 | 70.80 % |
| MFCC | MobileNetV3L | 70.31 % | 2.42 | 76.23 % | 2.25 | 77.26 % |
| MFCC | EfficientNetB1 | 48.01 % | 2.30 | 54.78 % | 2.09 | 63.82 % |
| MFCC | EfficientNetV2B1 | 42.18 % | 2.19 | 48.06 % | 2.03 | 58.14 % |
| MFCC | VGG16 | 54.02 % | 2.29 | 66.15 % | 2.08 | 65.63 % |
| MFCC | VGG19 | 52.37 % | 2.22 | 71.32 % | 1.98 | 68.73 % |

Among all model-feature combinations, the Mel Spectrogram paired with MobileNet yielded the highest performance, achieving accuracy rates of 79.87 %, 86.30 %, and 87.60 % on the training, validation, and test datasets, respectively. Notably, MobileNetV3L also demonstrated strong overall performance across multiple feature types, indicating the efficacy of lightweight yet expressive architectures for noise classification tasks.

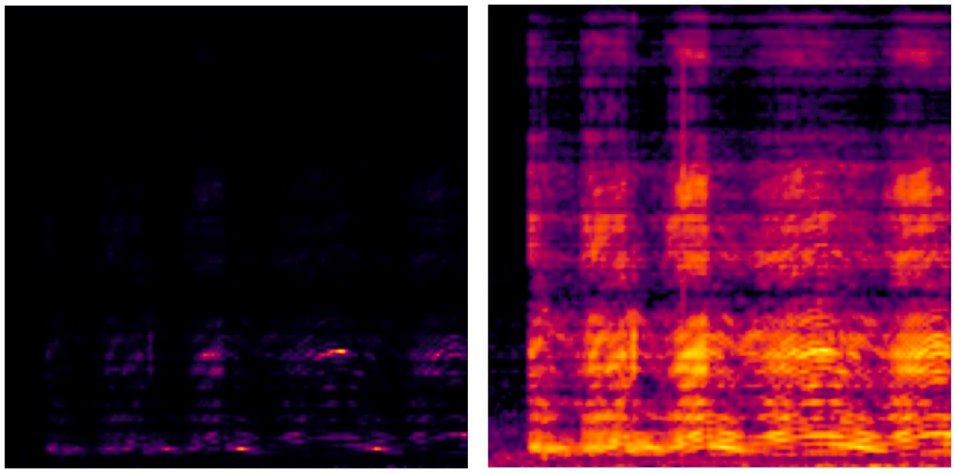
### *4.2. Experiment 2: Assess the impact of quantization on model performance, and determine feasibility for deployment on edge or sensing devices.*

We use TensorFlow Lite to quantize 24 models from Experiment 1. The comparison of the model accuracy of the test dataset and size before and after quantization is presented in Table 3. Quantization reduces model size by approximately four times, with a slight impact on accuracy. On average, accuracy decreases by 0.73 % with the highest reduction of 3.88 % under Spectrogram and MobileNetV3Small.

Despite quantization, the combination of Mel Spectrogram and MobileNet still attains the highest accuracy with 86.56 % accuracy, and a model size of 19.25 MB, making it the second smallest model after MobileNetV3Small. However, the performance of the MobileNetV3Small is below that of MobileNet. While it is feasible to run our best model on Raspberry Pi 3 Model B, which features a quad-core 1.2 GHz CPU and 1 GB RAM, it is impossible to fit the model directly on the ESP32 sensing device with a dual-core 240 MHz CPU, 512 KB RAM, and 4 MB flash memory. Even the smallest MobileNetV3Small model, at 10.08 MB, is still much larger than ESP32's memory capacity.

### *4.3. Experiment 3: Explore image-based and hierarchical models.*

To enhance the performance of our noise identification models, we explore alternative approaches. We implement two additional models:



(a) Original Image (b) With Decibel Conversion

**Fig. 1.** Mel spectrogram images.

**Table 3**
Comparison of original and quantized models under test dataset.

| Feature Extraction | Model Architecture | Test (Quantization) | | | |
|---|---|---|---|---|---|
| | | Original Size (MB) | Original Accuracy | Quantized Size (MB) | Quantized Accuracy |
| Spectrogram | MobileNet | 76.25 | 66.67 % | 19.25 | 65.89 % |
| Spectrogram | MobileNetV2 | 88.50 | 63.05 % | 22.42 | 63.05 % |
| Spectrogram | MobileNetV3S | 39.60 | 69.77 % | 10.08 | 65.89 % |
| Spectrogram | MobileNetV3L | 71.38 | 72.09 % | 18.15 | 74.16 % |
| Spectrogram | EfficientNetB1 | 104.88 | 47.80 % | 26.99 | 47.29 % |
| Spectrogram | EfficientNetV2B1 | 106.18 | 59.43 % | 27.38 | 60.72 % |
| Spectrogram | VGG16 | 88.18 | 70.28 % | 22.13 | 70.28 % |
| Spectrogram | VGG19 | 108.44 | 71.06 % | 27.21 | 71.06 % |
| Mel Spectrogram | MobileNet | 76.25 | 87.60 % | 19.25 | 86.56 % |
| Mel Spectrogram | MobileNetV2 | 88.50 | 79.84 % | 22.42 | 80.36 % |
| Mel Spectrogram | MobileNetV3S | 39.60 | 78.55 % | 10.08 | 76.49 % |
| Mel Spectrogram | MobileNetV3L | 71.38 | 82.43 % | 18.15 | 79.59 % |
| Mel Spectrogram | EfficientNetB1 | 104.88 | 67.96 % | 26.99 | 65.89 % |
| Mel Spectrogram | EfficientNetV2B1 | 106.18 | 68.73 % | 27.38 | 67.70 % |
| Mel Spectrogram | VGG16 | 88.18 | 72.87 % | 22.13 | 73.90 % |
| Mel Spectrogram | VGG19 | 108.44 | 79.84 % | 27.21 | 78.81 % |
| MFCC | MobileNet | 76.25 | 76.23 % | 19.25 | 73.39 % |
| MFCC | MobileNetV2 | 88.50 | 58.91 % | 22.42 | 56.85 % |
| MFCC | MobileNetV3S | 39.60 | 70.80 % | 10.08 | 69.25 % |
| MFCC | MobileNetV3L | 71.38 | 77.26 % | 18.15 | 77.52 % |
| MFCC | EfficientNetB1 | 104.88 | 63.82 % | 26.99 | 64.86 % |
| MFCC | EfficientNetV2B1 | 106.18 | 58.14 % | 27.38 | 56.33 % |
| MFCC | VGG16 | 88.18 | 65.63 % | 22.13 | 65.89 % |
| MFCC | VGG19 | 108.44 | 68.73 % | 27.21 | 68.22 % |

an *image-based* model and a *hierarchical* model. These new models are evaluated against our previous top-performing model, which is the Mel Spectrogram using MobileNet. In this section, we present the evaluation results, including accuracy, precision, recall, and F1 score, based on the test dataset.

**Image-Based Model:** Instead of developing the model directly with the transformed audio spectral information extracted from the feature extraction methods, we opt to train the model using images of Spectrogram, Mel Spectrogram, or MFCC. Hence, after feature extraction, the spectrum output is saved as a PNG image and then used as a model's input. As in Experiment 1, we implement and evaluate 24 model configurations. The most successful combination uses Mel Spectrogram and MobileNetV3Large, yielding an accuracy, precision, recall, and F1 score of 82.43 %, 0.83, 0.82, 0.82, respectively (see Table 4). We select this model for further refinement.

**Table 4**
Performance of image-based, single-level, and hierarchical models based on the test dataset.

| Model | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| Image-Based Model | 82.43 % | 0.83 | 0.82 | 0.82 |
| Image-Based Model with Decibel Conversion | 85.05 % | 0.86 | 0.85 | 0.85 |
| Single-Level Model (Exp.1) | 87.60 % | 0.88 | 0.88 | 0.88 |
| Hierarchical Model | 90.18 % | 0.91 | 0.90 | 0.90 |
| Audio Spectrogram Transformer | 87.83 % | 0.88 | 0.88 | 0.88 |
| Hierarchical Model (Quantized) | 86.30 % | 0.87 | 0.86 | 0.86 |

To improve the model, we apply the *decibel conversion* to convert the color representation unit in the Mel Spectrogram image from amplitude to decibels, enhancing visualization of the spectrum's dynamic range. Fig. 1(a) and (b) illustrate the original Mel Spectrogram image and the image after decibel conversion, respectively. Applying decibel conversion to input images helps increase the classification accuracy to 85.05 % as shown in Table 4. However, the image-based model performance remains slightly lower compared to the model using Mel Spectrogram and MobileNet with audio spectral input (from Experiment 1).

**Hierarchical Model:** From Experiment 1, we observe that misclassified cases often involve the car, train, and airplane engine classes as their audio signals exhibit similar characteristics. To address this issue, we develop a hierarchical model with two layers. The first layer differentiates noises with more distinct characteristics. We group audio with similar features, i.e., car, train, and airplane, into an engine class. Therefore, this model can distinguish seven audio classes: drilling, jackhammers, handsaws, street music, car horns, engine, and other urban sounds. The second layer narrows down the classification and focuses on identifying if the engine sound is a car, train, or airplane. We follow the model development process in Section 3. Mel Spectrogram is used for feature extraction, and all sub-models are implemented with the MobileNet architecture.

The results presented in Table 4 demonstrate that the hierarchical model outperforms both the image-based and single-level models from Experiment 1. It achieves an accuracy of 90.91 %, with a precision

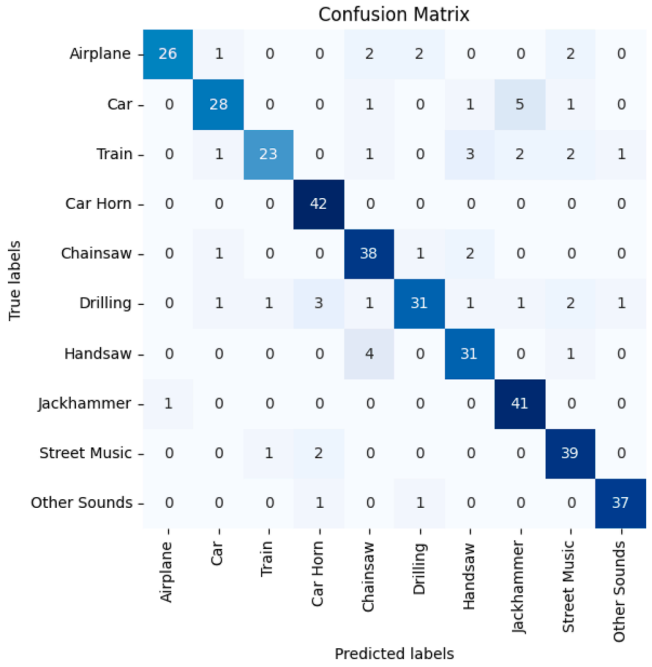



**Fig. 2.** Confusion matrix of hierarchical model.

of 0.91, a recall of 0.90, and an F1 score of 0.90. After quantization, the model's accuracy decreases to 86.3 %, but its size is significantly reduced from 152.45 MB to 38.49 MB. Furthermore, the hierarchical model was benchmarked against the state-of-the-art Audio Spectrogram Transformer (AST) (Gong et al., 2021). The comparative analysis reveals that the hierarchical model not only achieves marginally superior classification performance but also exhibits a considerably smaller model size, as the AST model requires 328.9 MB.

The confusion matrix illustrated in Fig. 2 provides an overview of the classification performance of the hierarchical model across the ten noise categories. Most classes show strong diagonal dominance, indicating high accuracy in classification. Distinctive sound categories such as Car Horn (42 out of 42 correct), Jackhammer (41 out of 42 correct), and Street Music (39 out of 41 correct) achieve nearly perfect recognition due to their unique acoustic characteristics.

In contrast, tool-related sounds like Drilling, Handsaw, and Chainsaw exhibit higher rates of misclassification, often getting confused with one another due to their overlapping spectral and temporal features. Similarly, transport-related sounds, such as Airplane, Car, and Train, also show cross-confusion, likely because of shared low-frequency components.

The heterogeneous class labeled Other Sounds demonstrates moderate accuracy, with errors dispersed across various categories. These results indicate that while the model performs well on acoustically distinctive classes, further refinement of features is necessary for categories with similar spectral signatures.

## 5. Acoustic sensing network development

Fig. 3 presents the overview of our acoustic sensing platform for noise monitoring and identification. The system consists of three tiers: (1) acoustic sensing devices, (2) edge devices, and (3) centralized back-end infrastructure. This design reduces the audio traffic loads and the computing burden on the central server.

### 5.1. Acoustic sensing devices

The sensing unit comprises an ESP32 microcontroller and a low-cost INMP441 MEMS microphone (Invensense, 2025) offering a cost-effective solution at approximately $12 USD per unit. ESP32 is connected to the INMP441 microphone through the I2S interface. Because the device handles multiple tasks, including sound level measurement, noise recording, and device management (i.e., authentication and configuration), we chose FreeRTOS to implement our embedded software for efficient task management. Note that our embedded software is adapted from Gunatilaka (2021). Before deploying the sensing unit, system admins must register the device through the web application. This process generates the device credentials for authentication, MQTT topics, and APIs to facilitate communication with the back-end system. This information must be installed on the sensing device before deployment.

Once the device is turned on, it sends an authentication request to the back-end server. If authentication is successful, the device will start the sound level measurement task, which samples audio data from the microphone and computes the A-weighted equivalent continuous sound level $L_{Aeq}$. $L_{Aeq}$ is then published to the server via the MQTT broker. Upon detecting a noise event, i.e., when a noise level exceeds 85 dB safety threshold (as defined in National Institute for Occupational Safety and Health, NIOSH), a noise recording task begins to capture an audio recording in the .wav format, and transmits the recording and its metadata to an edge device via an HTTP request for noise source classification. To achieve accurate sound level measurement, the microphone is configured to sample at 48 kHz and 32-bit depth. However, for noise recording, we downsample the audio to 16 kHz and 16-bit depth to reduce the audio file size and transmission overhead. While executing other tasks, the device continues to monitor device status queries and configuration requests from the back-end server by subscribing to device configuration MQTT topics. If a new configuration is successfully applied, the device will respond to the server with a confirmation message. Our embedded software occupies 1.14 MB of program storage, with 48.88 KB of RAM dedicated to global variables.

### 5.2. Edge devices

Raspberry Pi 3 Model B serves as an edge device since it has sufficient resources to run the classification model efficiently. Once the edge device receives a noise file from the sensing device, it initiates data preprocessing and feature extraction. The extracted audio data is then fed to the noise source identification model. We deploy a combination of the Mel Spectrogram and the MobileNet-based hierarchical model, which outperforms the other techniques. The edge device then transfers the classification output to the back-end system via an HTTP protocol. The system also offers an option to transfer the full noise recording file to the back-end server. This feature supports the collection of audio recordings, which is useful for future model development and noise analysis.

### 5.3. Centralized back-end infrastructure

On the server side, the web server is connected to the MQTT broker, the Google Firebase authentication service, and the Google Firestore. These three components provide communication and data storage infrastructure for our platform. MQTT, a lightweight publish-subscribe protocol, enables communication between the sensing devices and the back-end server. This protocol can efficiently transfer small sensor data (i.e., $L_{Aeq}$) and configuration commands because of its low overhead. Our work uses a cloud-based Mosquitto MQTT broker to manage message exchange between all MQTT clients. Moreover, the HTTP protocol is utilized for noise recording transfer and communication between an edge device and the web server, as HTTP is more suitable for large file transfers than MQTT. Our current implementation adopts Wi-Fi for data transmission. However, alternative wireless technologies such as LoRa, NB-IoT, 4G/5G, etc., could replace it if long-range communication is required. Google Firebase authentication provides a secure and easy-to-implement service for sensing device and web user authentication. All audio and device data are stored on Google Firestore, a cloud-based NoSQL database that offers scalable data storage.

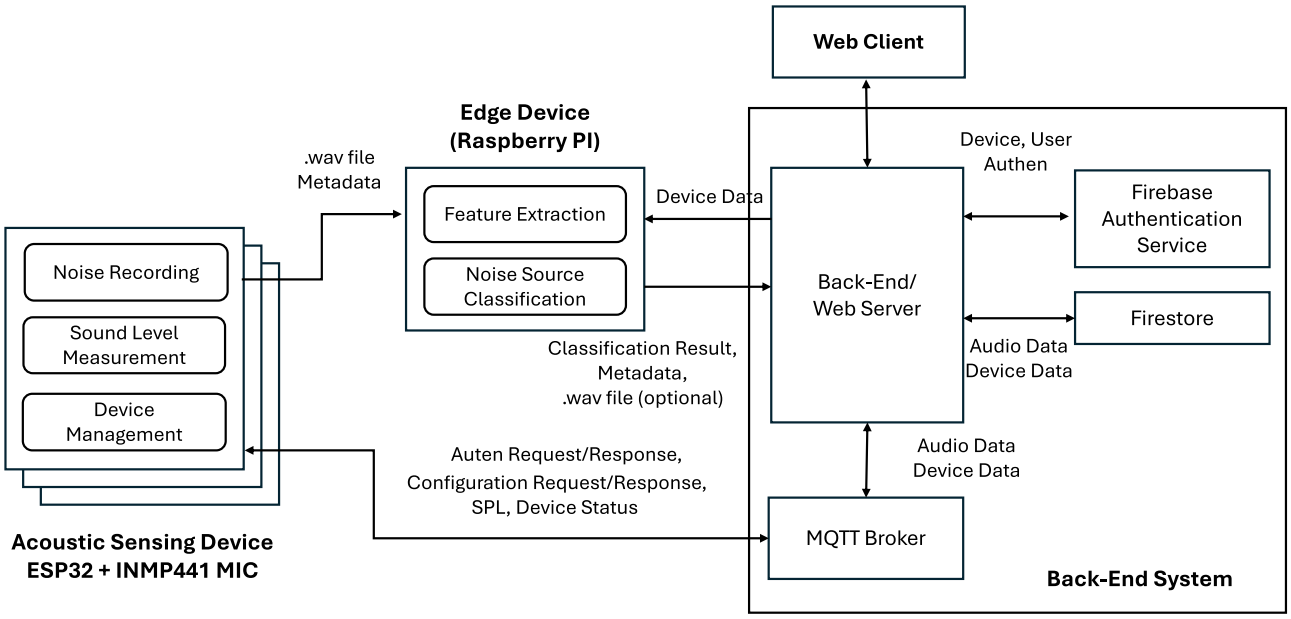



**Fig. 3.** Acoustic sensing system for noise monitoring and identification overview.

The back-end server functions as an IoT platform that handles audio data aggregation and device management. It also hosts a web server implemented with Node.js and the Express.js framework. The web dashboard is developed with React, a JavaScript library for the web front-end. It can be accessed through a web browser on both computers and mobile devices. Our web application consists of five main features:

- Real-Time Noise Monitoring: The web interface displays real-time noise levels and identified noise sources for each sensor location on a Google map. We use Firestore's live snapshot to push real-time updates to the dashboard as new noise data becomes available.
- Historical Noise Data Visualization: For historical noise data, we present it using a time-series graph and a noise event list. Users can filter this data by selecting a specific time range.
- Noise Recording Playback and Verification: If an edge device is configured to transmit a noise file along with the classification result to the server, users can play back the recording in the noise event list. This allows system admins to verify the noise source and correct the classification result if necessary.
- Complaint System: The system provides a feature that allows urban residents to submit noise-related complaints. This information is useful for correlating with the noise event automatically detected by our system and assessing the event's impact.
- Device Management: For device management, admins can register a device to the system before installation, and delete the devices that are no longer in use. They can also enable/disable the devices and update their configurations, such as sound level sampling period, noise recording length, noise event thresholds, etc. When an admin submits a new configuration, a configuration command is dispatched to the sensing device via the MQTT protocol. Moreover, the system provides a logging mechanism for monitoring the activities of the sensing device and the back-end components, which is useful for system management and troubleshooting.

Fig. 4(a)–(c) show examples of our web interfaces comprising the real-time noise monitoring map, the historical noise data page, and the device configuration page, respectively.

## 6. System evaluation

We conduct functionality tests to ensure all components of the system, the sensing device, the edge device, and the back-end infrastructure, operate correctly. The low-cost microphone is calibrated with the IEC 61,672 class-2 sound level meter (Instruments, 2025) based on (Gunatilaka, 2021). Then, we test the entire system pipeline. The sensing device continuously measures $L_{Aeq}$ and publishes this data to the central server via MQTT every five seconds. We simulate 100 unseen loud noise events to let the sensing device record the noise and transfer it to the Raspberry Pi. The edge device identifies the noise source and transmits the result to the back-end server. Audio data and events are then displayed on the web UI. All communication takes place over Wi-Fi networks. We assess the following aspects of the system.

**Inference Time and Model Accuracy**: We measure the feature extraction and inference time on the Raspberry Pi 3 Model B for both non-quantized and quantized models using 5-s recordings. On average, feature extraction takes 1.44 s, while the average inference times are 0.97 s and 0.7 s for the original and quantized models, respectively. Although the quantized model size is 3.9 times smaller than the original, its inference time improves by only 27.84 %. With the 5-s audio data obtained from the low-cost microphone, both the original and the quantized models obtain an accuracy of 86 %.

**Audio Length vs Accuracy**: In scenarios where high-data-rate Wi-Fi is unavailable, or a transition to lower-data-rate, long-range communication technologies (e.g., NB-IoT, LoRa, Sigfox) is necessary, sending audio files will incur significantly longer delay. To cut down these delays, an immediate solution is to reduce the size of the transmitted noise files. Therefore, our study examines the impact of different recording sizes on overall classification accuracy. As shown in Table 5, reducing the recording length decreases the file size; however, this negatively affects the model accuracy. With the original model, it may be feasible to reduce the recording duration to 4 s, as the classification accuracy does not experience a significant decline. Furthermore, the original model consistently achieves comparable or superior performance compared to the quantized models in all cases.

**Transmission Delay**: We focus on the delays in (1) transferring the noise file from the sensing devices to the Raspberry Pi, and (2) publishing sound level data to the back-end server. The delay depends on several factors, including transmission data rate (based on wireless technology in use), the data/file size, and network conditions. Based on our experiment, a 5-s recording with a file size of 160 KB has an av-




(a) Noise Monitoring Map (b) Historical Noise Data (c) Device Configuration

**Fig. 4.** Examples of noise monitoring web interfaces.

**Table 5**
Original and quantized model accuracy under different noise recording lengths.

| Recording Length | File Size | Model Accuracy |
|---|---|---|
| 5 seconds | 160 KB | Original: 86 %<br>Quantized: 86 % |
| 4 seconds | 125 KB | Original: 83 %<br>Quantized: 81 % |
| 3 seconds | 94 KB | Original: 74 %<br>Quantized: 73 % |
| 2 seconds | 63 KB | Original: 49 %<br>Quantized: 46 % |

erage transmission delay of 0.37 s. Note that the sensing device and the Raspberry Pi are located within the same Wi-Fi network. The average delay for transmitting the 200-byte sound level measurement message, containing $L_{Aeq}$ and its metadata, is 0.36 s. The delay in transferring sound level measurements is comparable to that of audio file transfers despite the significantly smaller data size of the former. This is because the sensing devices, the cloud-based MQTT broker, and the backend server are all situated on separate networks, introducing higher latency.

**Transmission Reliability**: We assess communication reliability using a data delivery ratio, a percentage of messages or files successfully delivered to the receiver. Our system achieves a 100 % data delivery ratio when transmitting noise files from a sensing device to the edge device. Additionally, all sound level measurements and classification results are received successfully by the back-end server.

## 7. Conclusion

In this paper, we design and implement an acoustic sensing network capable of continuously monitoring sound levels and identifying noise sources. Our work covers the development of an embedded system equipped with a low-cost microphone to measure and collect noise data, the use of an edge device for noise classification, and the implementation of the back-end server and infrastructure, which is essential for effective noise data and device management. We conduct thorough comparative studies to identify the best feature extraction technique and pre-trained CNN model for noise classification. The hierarchical model using Mel Spectrogram and MobileNet outperforms the other methods with an accuracy of 90.18 %, and is deployed on a Raspberry Pi edge device. In addition, evaluation results demonstrate that our system can reliably transmit audio data with small delays, and the Raspberry Pi can complete feature extraction and noise source classification on average within 2.5 s. Hence, this work shows the potential of using low-cost equipment and a pre-trained model to achieve a scalable and cost-effective solution to improve noise monitoring and control.

**Future Works:** While Wi-Fi technology offers high-bandwidth data transfer, it is inherently power-intensive and has a limited communication range. Using long-range and low-power technologies such as NB-IoT or LoRa would be more practical for large-scale deployment. However, these technologies could introduce significant communication delays, especially for audio recording transfer. One future solution is to implement the noise classification model directly on the sensing device. This can be achieved by developing an optimized model, specifically for microcontrollers, as deploying a pre-trained model on an ESP32 is not feasible. Deploying the model on the sensing devices would also eliminate the communication overhead between the sensing unit and the edge device. Additionally, incorporating more context (e.g., time of day/week, events, weather) could help improve noise classification and provide better insights for noise management.

Our current model is designed to detect only one dominant noise event per audio clip. However, in real-world scenarios, multiple noise events may overlap. To address this, we could transition to a model that can output multiple classes for each audio clip, which would require a dataset marked with multiple labels per segment and modifications to the pre-trained models. Alternatively, we could implement sound event detection (SED), which produces time-series outputs of detected events. This approach is more complex, as it requires datasets containing longer audio recordings with precise, time-stamped annotations for each noise event. SED provides a much more detailed output, including the specific start and end times of each sound event.

## CRediT authorship contribution statement

**Dolvara Gunatilaka:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization, Supervision, Project administration; **Wudhichart Sawangphol:** Conceptualization, Methodology, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing; **Thanakorn Charoenritthitham:** Methodology, Software, Validation, Formal analysis, Investigation, Data curation; **Thanawat Kanjanapoo:** Methodology, Software, Validation, Formal analysis, Investigation, Data curation; **Teerapat Burasotikul:** Software, Investigation, Data curation; **Kittikawin Pongprasit:** Software, Investigation.

## Data availability

Data will be made available on request.

## Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Dolvara Gunatilaka reports financial support, administrative support, and article publishing charges were provided by Mahidol University. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This research project is supported by Mahidol University (No. A44/2563).

## References


Abdoli, S., Cardinal, P., & Lameiras Koerich, A. (2019). End-to-end environmental sound classification using a 1D convolutional neural network. *Expert Systems with Applications*, *136*, 252–263. https://doi.org/10.1016/j.eswa.2019.06.040

AbeBer, J., Gotze, M., Kuhnlenz, S., Grafe, R., Kuhn, C., ClauB, T., & Lukashevich, H. (2018). A distributed sensor network for monitoring noise level and noise sources in urban environments. In *2018 IEEE 6th international conference on future internet of things and cloud (FICLOUD)* (pp. 318–324). https://doi.org/10.1109/FiCloud.2018.00053

Aksoy, B., Usta, U., Karadağ, G., Kaya, A. R., & Ömür, M. (2022). Classification of environmental sounds with deep learning. *Advances in Artificial Intelligence Research*, *2*(1), 20-28.

Alsouda, Y., Pllana, S., & Kurti, A. (2019). Iot-based urban noise identification using machine learning: Performance of SVM, KNN, bagging, and random forest. In *Proceedings of the international conference on omni-layer intelligent systems* COINS '19 (p. 62-67). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3312614.3312631

Alías, F., & Alsina-Pagès, R. M. (2019). Review of wireless acoustic sensor networks for environmental noise monitoring in smart cities. *Journal of Sensors*, *2019*(1), 7634860. https://doi.org/10.1155/2019/7634860

Arce, P., Salvo, D., Piñero, G., & Gonzalez, A. (2021). Fiware based low-cost wireless acoustic sensor network for monitoring and classification of urban soundscape. *Computer Networks*, *196*, 108199. https://doi.org/10.1016/j.comnet.2021.108199

Ashurov, A., Yi, Z., Liu, H., Yu, Z., & Li, M. (2024). Concatenation-based pre-trained convolutional neural networks using attention mechanism for environmental sound classification. *Applied Acoustics*, *216*, 109759. https://doi.org/10.1016/j.apacoust.2023.109759

Ashurov, A., Zhou, Y., Shi, L., Zhao, Y., & Liu, H. (2022). Environmental sound classification based on transfer-learning techniques with multiple optimizers. *Electronics*, *11*(15). https://doi.org/10.3390/electronics11152279

Bansal, A., & Garg, N. K. (2022). Environmental sound classification: A descriptive review of the literature. *Intelligent Systems with Applications*, *16*, 200115. https://doi.org/10.1016/j.iswa.2022.200115

Bartalucci, C., Borchi, F., Carfagni, M., Furferi, R., Governi, L., SILVAGGIO, R., Curcuruto, S., & NENCINI, L. (2018). The smart noise monitoring system implemented in the frame of the life MONZA project. In *Euronoise* (pp. 783–788).

Basner, M., Babisch, W., Davis, A., Brink, M., Clark, C., Janssen, S., & Stansfeld, S. (2014). Auditory and non-auditory effects of noise on health. *The Lancet*, *383*(9925), 1325–1332. https://doi.org/10.1016/S0140-6736(13)61613-X

Bello, J. P., Silva, C., Nov, O., Dubois, R. L., Arora, A., Salamon, J., Mydlarz, C., & Doraiswamy, H. (2019). Sonyc: A system for monitoring, analyzing, and mitigating urban noise pollution. *Communications of the ACM*, *62*(2), 68-77. https://doi.org/10.1145/3224204

Bellucci, P., Peruzzi, L., & Zambon, G. (2017). Life dynamap project: The case study of rome. *Applied Acoustics*, *117*, 193–206. Acoustics in Smart Cities. https://doi.org/10.1016/j.apacoust.2016.04.022

Fernandez-Prieto, J.-A., Cañada-Bago, J., & Gadeo-Martos, M.-A. (2020). Wireless acoustic sensor nodes for noise monitoring in the city of linares (jaén). *Sensors*, *20*(1). https://doi.org/10.3390/s20010124

Foundation, F. (a). FIWARE: A framework of open source platform components to accelerate the development of smart solutions. https://www.fiware.org/about-us/. Accessed: 2025-06-19.

Gong, Y., Chung, Y.-A., & Glass, J. (2021). AST: Audio spectrogram transformer. In *Proc. interspeech 2021* (pp. 571–575). https://doi.org/10.21437/Interspeech.2021-698

Gunatilaka, D. (2021). An iot-enabled acoustic sensing platform for noise pollution monitoring. In *2021 IEEE 12th annual ubiquitous computing, electronics & mobile communication conference (UEMCON)* (pp. 0383–0389). https://doi.org/10.1109/UEMCON53757.2021.9666534

Instruments, L. (b). Sound -level meter LUTRON SL-4023SD. https://www.lutroninstruments.eu/sound--noise/sound-level-meter-lutron-sl-4023sd/. Accessed: 2025-06-16.

Invensense (c). INMP441 microphone specification. https://invensense.tdk.com/wp-content/uploads/2015/02/INMP441.pdf. Accessed: 2025-06-14.

Lamrini, M., Chkouri, M. Y., & Touhafi, A. (2023). Evaluating the performance of pre-trained convolutional neural network for audio classification on embedded systems for anomaly detection in smart cities. *Sensors*, *23*(13). https://doi.org/10.3390/s23136227

Maijala, P., Shuyang, Z., Heittola, T., & Virtanen, T. (2018). Environmental noise monitoring using source classification in sensors. *Applied Acoustics*, *129*, 258–267. https://doi.org/10.1016/j.apacoust.2017.08.006

Mohaimenuzzaman, M., Bergmeir, C., West, I., & Meyer, B. (2023). Environmental sound classification on the edge: A pipeline for deep acoustic networks on extremely resource-constrained devices. *Pattern Recognition*, *133*, 109025. https://doi.org/10.1016/j.patcog.2022.109025

Mohamed, A.-M. O., Paleologos, E. K., & Howari, F. M. (2021). Chapter 19 - noise pollution and its impact on human health and the environment. In *Pollution assessment for sustainable practices in applied sciences and engineering* (pp. 975–1026). Butterworth-Heinemann. https://doi.org/10.1016/B978-0-12-809582-9.00019-0

Mydlarz, C., Salamon, J., & Bello, J. P. (2017). The implementation of low-cost urban acoustic monitoring devices. *Applied Acoustics*, *117*, 207–218. https://doi.org/10.1016/j.apacoust.2016.06.010

National Institute for Occupational Safety and Health (NIOSH), & Noise and hearing loss. https://www.cdc.gov/niosh/index.html. Accessed: 2025-06-14.

Peng, B., Abdulla, W. H., & Wang, K. I.-K. (2023). Urban noise monitoring using edge computing with CNN-LSTM on jetson nano. In *2023 Asia pacific signal and information processing association annual summit and conference (APSIPA ASC)* (pp. 2244–2250). https://doi.org/10.1109/APSIPAASC58517.2023.10317387

Picaut, J., Can, A., Fortin, N., Ardouin, J., & Lagrange, M. (2020). Low-cost sensors for urban noise monitoring networks-a literature review. *Sensors*, *20*(8). https://doi.org/10.3390/s20082256

Piczak, K. J. (2015a). Environmental sound classification with convolutional neural networks. In *2015 IEEE 25th international workshop on machine learning for signal processing (MLSP)* (pp. 1–6). https://doi.org/10.1109/MLSP.2015.7324337

Piczak, K. J. (2015b). ESC: Dataset for environmental sound classification. In *Proceedings of the 23rd ACM international conference on multimedia* MM '15 (p. 1015-1018). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/2733373.2806390

Rainham, D. (2016). A wireless sensor network for urban environmental health monitoring: Urbansense. *IOP Conference Series: Earth and Environmental Science*, *34*(1), 012028. https://doi.org/10.1088/1755-1315/34/1/012028

Salamon, J., & Bello, J. P. (2017). Deep convolutional neural networks and data augmentation for environmental sound classification. *IEEE Signal Processing Letters*, *24*(3), 279–283. https://doi.org/10.1109/LSP.2017.2657381

Salamon, J., Jacoby, C., & Bello, J. P. (2014). A dataset and taxonomy for urban sound research. In *Proceedings of the 22nd ACM international conference on multimedia* MM '14 (p. 1041-1044). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/2647868.2655045

Shah, S. K., Tariq, Z., & Lee, Y. (2019). Iot based urban noise monitoring in deep learning using historical reports. In *2019 IEEE International conference on big data (big data)* (pp. 4179–4184). https://doi.org/10.1109/BigData47090.2019.9006176

Siamwala, A., Lochhead, Z., & Abdulla, W. (2019). Environmental noise monitoring using distributed iot sensor nodes. In *2019 International conference on electronics, information, and communication (ICEIC)* (pp. 1–10). https://doi.org/10.23919/ELINFOCOM.2019.8706473

da Silva, B., Axel, W. H., Braeken, A., & Touhafi, A. (2019). Evaluation of classical machine learning techniques towards urban sound recognition on embedded systems. *Applied Sciences*, *9*(18). https://doi.org/10.3390/app9183885

Tsalera, E., Papadakis, A., & Samarakou, M. (2021). Comparison of pre-trained CNNs for audio classification using transfer learning. *Journal of Sensor and Actuator Networks*, *10*(4). https://doi.org/10.54569/aair.1017801

Vidaña-Vila, E., Navarro, J., Borda-Fortuny, C., Stowell, D., & Alsina-Pagès, R. M. (2020). Low-cost distributed acoustic sensor network for real-time urban sound monitoring. *Electronics*, *9*(12). https://doi.org/10.3390/electronics9122119

Vujošević, L., & Dukanović, S. (2021). Deep learning-based classification of environmental sounds. In *2021 25th international conference on information technology (IT)* (pp. 1–4). https://doi.org/10.1109/IT51528.2021.9390124

Wang, Y., Feng, C., & Anderson, D. V. (2021). A multi-channel temporal attention convolutional neural network model for environmental sound classification. In *ICASSP 2021 - 2021 ieee international conference on acoustics, speech and signal processing (ICASSP)* (pp. 930–934). https://doi.org/10.1109/ICASSP39728.2021.9413498

Yang, C., Gan, X., Peng, A., & Yuan, X. (2023). ResNet based on multi-feature attention mechanism for sound classification in noisy environments. *Sustainability*, *15*(14). https://doi.org/10.3390/su151410762